COMPLETE MASTER GUIDE

MySQL

& Database Management

From Database Fundamentals to Advanced Administration

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

WHAT THIS COMPLETE GUIDE COVERS

TABLE OF CONTENTS

Chapter 1 — What Is a Database?

A database is an organised, structured collection of data stored electronically on a computer system. It allows data to be easily stored, retrieved, updated, and managed. Without databases, every website, app, and business system would have to store data in unorganised text files — making it nearly impossible to search, sort, or protect.

Think of a database like a giant, intelligent filing cabinet. Instead of hunting through hundreds of paper files, you can ask it a question in seconds: 'Give me all students from Nyeri who scored above 80 in JavaScript.'

1.1 Why Databases Are Essential

Who Uses Databases	How They Use Them
Websites & Apps	Every user account, product listing, blog post, and transaction lives in a database.
Business Operations	Sales records, payroll, inventory, customer data — all managed in databases.
Schools & Institutions	Student records, exam results, timetables, fee payments.
Government Services	Tax records (KRA iTax), citizen IDs, land registry, health records.
Banking & Finance	Every Mpesa transaction, bank transfer, and account balance — all in databases.
Healthcare	Patient records, prescriptions, lab results, appointment booking.

1.2 Types of Database Management Systems (DBMS)

DBMS Type	Description & Examples
Relational (RDBMS)	Data stored in tables with rows & columns. Uses SQL. Most common type. Examples: MySQL, PostgreSQL, Oracle, SQL Server, SQLite.
NoSQL	Non-tabular storage — documents, key-value pairs, graphs. Examples: MongoDB (documents), Redis (key-value), Cassandra (wide-column), Neo4j (graph).
In-Memory	Data stored in RAM for ultra-fast access. Examples: Redis, Memcached.
Cloud Databases	Database as a Service (DBaaS). Examples: Amazon RDS, Google Cloud SQL, PlanetScale, Supabase.
Time-Series	Optimised for time-stamped data. Examples: InfluxDB, TimescaleDB. Used in IoT, monitoring.
NewSQL	Combines SQL reliability with NoSQL scalability. Examples: CockroachDB, Google Spanner.

1.3 Why MySQL?

- ❖ Free and open-source — zero licensing cost.
- ❖ The most widely used RDBMS in the world for web applications.
- ❖ Powers WordPress, Facebook (originally), YouTube, Twitter, Wikipedia.
- ❖ Excellent PHP integration — the PHP + MySQL combination is the backbone of millions of Kenyan and global websites.
- ❖ Easy to learn with tools like phpMyAdmin providing a visual interface.
- ❖ Available on Windows, Linux, and Mac — runs on XAMPP, WAMP, MAMP, Laragon.
- ❖ Backed by Oracle — well-maintained, reliable, and battle-tested for decades.

DATABASE vs SPREADSHEET — Know the Difference

A spreadsheet (Excel, Google Sheets) is great for small, flat data with manual analysis.

A database is essential when you need:

- More than a few thousand records
- Multiple users reading/writing simultaneously
- Complex relationships between different types of data
- Automatic data integrity rules (no duplicate emails, no negative prices)
- Programmatic access from websites and applications
- Data security and access control by role

Chapter 2 — Relational Database Theory

A relational database stores data in tables. Each table represents one entity — like students, courses, or payments. Tables are connected to each other through relationships using keys.

2.1 Core Concepts

Term	Definition
Table (Relation)	A set of rows and columns representing one entity. Like a spreadsheet tab. Example: a 'students' table.
Row (Record/Tuple)	A single entry in a table — one complete data item. One student = one row.
Column (Field/Attribute)	A specific attribute of the entity. 'name', 'email', 'age' are all columns.
Primary Key (PK)	A column (or combination) that uniquely identifies each row. No two rows can have the same PK. Usually an auto-increment integer 'id'.
Foreign Key (FK)	A column in one table that references the Primary Key of another table. This creates a relationship between tables.
Schema	The structure/design of a database — all its tables, columns, data types, and relationships.
Query	A request for data, written in SQL. A SELECT query asks for data; INSERT adds data; UPDATE changes it; DELETE removes it.
Index	A data structure that speeds up data retrieval. Like a book's index — helps find rows without scanning the whole table.
Constraint	A rule enforced on data. Examples: NOT NULL (field cannot be empty), UNIQUE (no duplicates allowed).
Transaction	A unit of work containing one or more SQL statements that must all succeed or all fail together.

2.2 Types of Relationships

Relationship Type	Description & Example
One-to-One (1:1)	One record in Table A relates to exactly one record in Table B. Example: One user → one profile.
One-to-Many (1:N)	One record in Table A relates to many records in Table B. Example: One student → many exam results.
Many-to-Many (M:N)	Many records in Table A relate to many records in Table B. Requires a JUNCTION (pivot) table. Example: Students enrol in many courses; courses have many students.

2.3 Understanding Primary & Foreign Keys

```
-- students table
-- 'id' is the PRIMARY KEY — unique identifier for each student
+----+---------------+-------------------------+----------+
| id | name          | email                   | city     |
+----+---------------+-------------------------+----------+
|  1 | Lucky Nakola  | lucky@example.com       | Nyeri    |
|  2 | Alice Wanjiru | alice@example.com       | Nairobi  |
|  3 | Bob Kamau     | bob@example.com         | Kisumu   |
+----+---------------+-------------------------+----------+

-- enrollments table
-- 'student_id' is a FOREIGN KEY referencing students.id
-- 'course_id'  is a FOREIGN KEY referencing courses.id
+----+------------+-----------+------------+-----------+
| id | student_id | course_id | enrolled   | grade     |
+----+------------+-----------+------------+-----------+
|  1 |          1 |         3 | 2025-01-10 | A         |
|  2 |          2 |         3 | 2025-01-10 | B         |
|  3 |          1 |         5 | 2025-02-01 | A         |
+----+------------+-----------+------------+-----------+
-- Row 1: student 1 (Lucky) is enrolled in course 3
-- Row 3: student 1 (Lucky) is ALSO enrolled in course 5
-- This is a Many-to-Many relationship via a junction table
```

Chapter 3 — MySQL Installation, Tools & the Environment

3.1 Setting Up MySQL with XAMPP

1. Download XAMPP from apachefriends.org and install it.
2. Open the XAMPP Control Panel and click START next to MySQL.
3. Open your browser and go to: http://localhost/phpmyadmin
4. You are now inside phpMyAdmin — the visual MySQL administration tool.
5. Alternatively, open a terminal and type: mysql -u root -p to access MySQL via command line.

3.2 MySQL Tools Reference

Tool	Description & When to Use
phpMyAdmin	Web-based GUI for managing MySQL databases. Create tables, run queries, import/export data. Perfect for beginners.
MySQL Workbench	Official MySQL desktop application. Visual ER diagram designer, query editor, server administration, performance monitoring.
MySQL CLI	Command-line interface. Type 'mysql -u root -p' in terminal. Full power — no GUI needed. Used in production servers.
DBeaver	Free universal database client. Supports MySQL, PostgreSQL, SQLite, and 80+ databases. Great alternative to Workbench.
TablePlus	Modern, fast GUI for multiple databases. Native app — feels very clean and professional.
HeidiSQL	Lightweight Windows MySQL client. Fast, simple, free.
VS Code + Extension	The 'MySQL' or 'Database Client' VS Code extensions let you query databases directly from your editor.

3.3 Essential MySQL CLI Commands

```
-- Connect to MySQL
mysql -u root -p
mysql -u root -p school_db          -- Connect directly to a database
mysql -h 127.0.0.1 -P 3306 -u root -p  -- Explicit host and port

-- Once inside MySQL CLI:
SHOW DATABASES;                     -- List all databases
USE school_db;                      -- Switch to a database
SHOW TABLES;                        -- List all tables in current
database
DESCRIBE students;                  -- Show structure of a table
DESC students;                      -- Same as DESCRIBE (shorthand)
SHOW CREATE TABLE students;         -- Show full CREATE TABLE
statement
SHOW COLUMNS FROM students;         -- Show column details
SHOW INDEXES FROM students;         -- Show all indexes on a table
STATUS;                             -- Show MySQL server status
SELECT VERSION();                   -- MySQL version
SELECT USER();                      -- Current user
SELECT DATABASE();                  -- Current database
EXIT;                               -- Leave MySQL CLI
QUIT;                               -- Same as EXIT
```

3.4 SQL Language Categories

Category	What It Controls & Commands
DDL — Data Definition Language	Commands that define and modify the database structure. CREATE, ALTER, DROP, TRUNCATE, RENAME.
DML — Data Manipulation Language	Commands that work with the data inside tables. INSERT, SELECT, UPDATE, DELETE.
DCL — Data Control Language	Commands that manage permissions and access. GRANT, REVOKE.
TCL — Transaction Control Language	Commands that manage transactions. COMMIT, ROLLBACK, SAVEPOINT.
DQL — Data Query Language	Some classify SELECT separately as DQL — it is purely for querying data.

Chapter 4 — SQL DDL: CREATE, ALTER, DROP

DDL (Data Definition Language) commands define the structure of your database — creating databases and tables, modifying their structure, and removing them. These commands change the schema, not the data.

4.1 CREATE DATABASE & USE

```
-- Create a new database
CREATE DATABASE school_db;

-- Create with character set (best practice — supports all languages & emojis)
CREATE DATABASE school_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Create only if it does not already exist (prevents errors)
CREATE DATABASE IF NOT EXISTS school_db;

-- Switch to the database you want to work in
USE school_db;

-- List all databases
SHOW DATABASES;

-- Delete a database PERMANENTLY (all tables and data gone)
DROP DATABASE school_db;
DROP DATABASE IF EXISTS school_db;
```

4.2 CREATE TABLE — The Full Syntax

```
CREATE TABLE students (
  id           INT           NOT NULL AUTO_INCREMENT,
  student_no   VARCHAR(20)   NOT NULL UNIQUE,
  full_name    VARCHAR(100)  NOT NULL,
  email        VARCHAR(150)  NOT NULL UNIQUE,
  phone        VARCHAR(20)   NULL,
  date_of_birth DATE         NULL,
  gender       ENUM('Male','Female','Other') NOT NULL,
  city         VARCHAR(80)   DEFAULT 'Nairobi',
  is_active    TINYINT(1)    NOT NULL DEFAULT 1,
  created_at   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP
                             ON UPDATE CURRENT_TIMESTAMP,
```

```
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- Table with a FOREIGN KEY constraint
CREATE TABLE enrollments (
  id         INT       NOT NULL AUTO_INCREMENT,
  student_id INT       NOT NULL,
  course_id  INT       NOT NULL,
  enrolled   DATE      NOT NULL DEFAULT (CURRENT_DATE),
  grade      CHAR(2)   NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

  PRIMARY KEY (id),

  -- Foreign key: student_id must exist in students.id
  CONSTRAINT fk_enrollment_student
    FOREIGN KEY (student_id) REFERENCES students(id)
    ON DELETE CASCADE       -- If student is deleted, delete their enrollments too
    ON UPDATE CASCADE,      -- If student id changes, update here too

  -- Foreign key: course_id must exist in courses.id
  CONSTRAINT fk_enrollment_course
    FOREIGN KEY (course_id) REFERENCES courses(id)
    ON DELETE RESTRICT      -- Prevent deleting a course that has enrollments
    ON UPDATE CASCADE,

  -- Prevent a student from enrolling in the same course twice
  UNIQUE KEY uq_student_course (student_id, course_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

4.3 ALTER TABLE — Modify Structure

```
-- Add a new column
ALTER TABLE students ADD COLUMN profile_photo VARCHAR(255) NULL AFTER email;

-- Add column at the beginning
ALTER TABLE students ADD COLUMN uuid CHAR(36) NOT NULL FIRST;

-- Modify a column's data type or constraints
ALTER TABLE students MODIFY COLUMN phone VARCHAR(30) NOT NULL;

-- Rename a column (MySQL 5.7+)
ALTER TABLE students RENAME COLUMN full_name TO name;

-- Change column name AND type
ALTER TABLE students CHANGE COLUMN city location VARCHAR(100) DEFAULT 'Kenya';

-- Drop (delete) a column
ALTER TABLE students DROP COLUMN profile_photo;
```

```
-- Add an index
ALTER TABLE students ADD INDEX idx_city (city);

-- Add a UNIQUE constraint
ALTER TABLE students ADD CONSTRAINT uq_email UNIQUE (email);

-- Add a FOREIGN KEY
ALTER TABLE enrollments
  ADD CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES students(id);

-- Rename a table
ALTER TABLE students RENAME TO school_students;
RENAME TABLE school_students TO students;   -- Alternative syntax

-- TRUNCATE — delete ALL rows but keep the table structure
TRUNCATE TABLE enrollments;

-- DROP — delete the entire table (structure + data)
DROP TABLE IF EXISTS temp_data;
```

Chapter 5 — DML: INSERT, SELECT, UPDATE, DELETE

DML (Data Manipulation Language) is where most of your day-to-day SQL work happens. These four commands — INSERT, SELECT, UPDATE, DELETE — are the foundation of every database-driven application.

5.1 INSERT — Adding Data

```
-- Insert a single row
INSERT INTO students (student_no, full_name, email, gender, city)
VALUES ('STU001', 'Lucky Nakola', 'lucky@example.com', 'Male', 'Nyeri');

-- Insert multiple rows in one statement (much faster than separate inserts)
INSERT INTO students (student_no, full_name, email, gender, city) VALUES
  ('STU002', 'Alice Wanjiru', 'alice@example.com', 'Female', 'Nairobi'),
  ('STU003', 'Bob Kamau',    'bob@example.com',   'Male',   'Kisumu'),
  ('STU004', 'Carol Odhiambo','carol@example.com','Female', 'Mombasa'),
  ('STU005', 'David Mutua',  'david@example.com', 'Male',   'Nairobi');

-- Get the AUTO_INCREMENT id of the last inserted row
SELECT LAST_INSERT_ID();

-- INSERT IGNORE — skip rows that would cause a duplicate key error
INSERT IGNORE INTO students (student_no, full_name, email, gender)
VALUES ('STU001', 'Duplicate', 'lucky@example.com', 'Male');
-- This is silently ignored because STU001 already exists

-- INSERT ... ON DUPLICATE KEY UPDATE (upsert)
INSERT INTO students (student_no, full_name, email, gender)
VALUES ('STU001', 'Lucky Nakola Updated', 'lucky@example.com', 'Male')
ON DUPLICATE KEY UPDATE
  full_name = VALUES(full_name),
  updated_at = CURRENT_TIMESTAMP;

-- INSERT ... SELECT — copy data from another table
INSERT INTO students_archive (student_no, full_name, email)
SELECT student_no, full_name, email
FROM students
WHERE is_active = 0;
```

5.2 SELECT — Reading Data

```
-- Select ALL columns from a table
SELECT * FROM students;

-- Select specific columns only (always prefer this over SELECT *)
SELECT id, full_name, email, city FROM students;

-- Column aliases — rename output columns
SELECT
  full_name  AS 'Student Name',
  email      AS 'Email Address',
  city       AS 'Location'
FROM students;

-- Select with expression
SELECT
  full_name,
  UPPER(email)                      AS email_upper,
  DATEDIFF(NOW(), created_at)       AS days_enrolled,
  CONCAT(full_name, ' — ', city)    AS name_city
FROM students;

-- Select DISTINCT values (no duplicates)
SELECT DISTINCT city FROM students;

-- Count rows
SELECT COUNT(*) AS total_students FROM students;
SELECT COUNT(DISTINCT city) AS unique_cities FROM students;
```

5.3 UPDATE — Modifying Data

```
-- Update a single row by primary key (ALWAYS use WHERE with UPDATE)
UPDATE students
SET city = 'Nakuru', phone = '+254712345678'
WHERE id = 1;

-- Update multiple rows matching a condition
UPDATE students
SET is_active = 0
WHERE city = 'Kisumu' AND created_at < '2024-01-01';

-- Update with calculation
UPDATE products
SET price = price * 1.10,         -- Increase price by 10%
    updated_at = CURRENT_TIMESTAMP
WHERE category = 'Electronics';

-- Update using a subquery
UPDATE students
```

```
SET city = (SELECT city FROM cities WHERE city_code = 'NBI')
WHERE id = 5;

-- ⚠  ALWAYS use WHERE with UPDATE!
-- UPDATE students SET is_active = 0;  -- This disables ALL students!

-- Check how many rows would be affected before updating
SELECT COUNT(*) FROM students WHERE city = 'Kisumu';
-- Then run the UPDATE
```

5.4 DELETE — Removing Data

```
-- Delete a single row (ALWAYS use WHERE with DELETE)
DELETE FROM students WHERE id = 10;

-- Delete multiple rows matching a condition
DELETE FROM enrollments WHERE grade = 'F' AND enrolled < '2024-01-01';

-- Delete with LIMIT (useful for bulk deletion in batches)
DELETE FROM logs WHERE created_at < '2024-01-01' LIMIT 1000;

-- Delete rows based on another table (using subquery)
DELETE FROM enrollments
WHERE student_id IN (
  SELECT id FROM students WHERE is_active = 0
);

-- ⚠  ALWAYS use WHERE with DELETE!
-- DELETE FROM students;  -- This deletes EVERY student row!

-- TRUNCATE vs DELETE
-- TRUNCATE: removes all rows instantly, resets AUTO_INCREMENT, cannot be rolled
back
TRUNCATE TABLE session_logs;

-- DELETE: row-by-row, triggers fire, can be rolled back in a transaction
DELETE FROM session_logs;  -- Slower but safer
```

THE GOLDEN RULE: ALWAYS USE WHERE WITH UPDATE AND DELETE

Running UPDATE or DELETE without a WHERE clause affects EVERY ROW in the table. This is one of the most common and devastating mistakes in SQL.

Best practice before any UPDATE or DELETE:

1. First run a SELECT with the same WHERE clause to see which rows will be affected.
2. Wrap in a transaction so you can ROLLBACK if something goes wrong.
3. Take a backup before large bulk operations.

Chapter 6 — Advanced SELECT: Filtering, Sorting & Limiting

6.1 WHERE Clause — Filtering Rows

```
-- Basic comparisons
SELECT * FROM students WHERE city = 'Nairobi';
SELECT * FROM students WHERE age > 18;
SELECT * FROM students WHERE age BETWEEN 18 AND 25;

-- Multiple conditions
SELECT * FROM students WHERE city = 'Nairobi' AND is_active = 1;
SELECT * FROM students WHERE city = 'Nairobi' OR city = 'Nyeri';

-- IN — check if value is in a list
SELECT * FROM students WHERE city IN ('Nairobi', 'Nyeri', 'Mombasa');
SELECT * FROM students WHERE city NOT IN ('Nairobi', 'Nairobi');

-- LIKE — pattern matching
SELECT * FROM students WHERE full_name LIKE 'Lucky%';    -- Starts with Lucky
SELECT * FROM students WHERE email LIKE '%@gmail.com';   -- Ends with @gmail.com
SELECT * FROM students WHERE full_name LIKE '%Nakola%';  -- Contains Nakola
SELECT * FROM students WHERE student_no LIKE 'STU_01';   -- _ matches any single
char

-- IS NULL and IS NOT NULL
SELECT * FROM students WHERE phone IS NULL;
SELECT * FROM students WHERE phone IS NOT NULL;

-- BETWEEN (inclusive on both ends)
SELECT * FROM orders WHERE order_date BETWEEN '2025-01-01' AND '2025-12-31';
SELECT * FROM products WHERE price BETWEEN 500 AND 5000;
```

6.2 ORDER BY — Sorting Results

```
-- Sort alphabetically by name (ASC = ascending, default)
SELECT * FROM students ORDER BY full_name ASC;

-- Sort by newest first (DESC = descending)
SELECT * FROM students ORDER BY created_at DESC;

-- Sort by multiple columns
SELECT * FROM students ORDER BY city ASC, full_name ASC;

-- Sort by column position (not recommended — but valid)
SELECT full_name, city, created_at FROM students ORDER BY 3 DESC;
```

```
-- Sort by expression
SELECT full_name, city FROM students ORDER BY CHAR_LENGTH(full_name) DESC;

-- Order with NULL handling (NULLs appear last in DESC, first in ASC)
SELECT * FROM products ORDER BY price IS NULL, price ASC;
```

6.3 LIMIT & OFFSET — Pagination

```
-- Get first 10 rows
SELECT * FROM students LIMIT 10;

-- Get rows 11-20 (page 2 with 10 per page)
SELECT * FROM students LIMIT 10 OFFSET 10;
SELECT * FROM students LIMIT 10, 10;   -- Shorthand: LIMIT offset, count

-- Pagination formula
-- page 1: LIMIT 10 OFFSET 0   (rows 1-10)
-- page 2: LIMIT 10 OFFSET 10  (rows 11-20)
-- page 3: LIMIT 10 OFFSET 20  (rows 21-30)
-- Formula: OFFSET = (page_number - 1) * rows_per_page

-- Get the 5 most recent students
SELECT * FROM students ORDER BY created_at DESC LIMIT 5;

-- Get the top 3 highest scoring results
SELECT student_id, score FROM exam_results ORDER BY score DESC LIMIT 3;
```

6.4 CASE Expression — Conditional Output

```
-- CASE works like if/elseif/else inside a SELECT
SELECT
  full_name,
  score,
  CASE
    WHEN score >= 90 THEN 'A — Distinction'
    WHEN score >= 75 THEN 'B — Credit'
    WHEN score >= 60 THEN 'C — Pass'
    WHEN score >= 50 THEN 'D — Borderline'
    ELSE                  'F — Fail'
  END AS grade,
  CASE WHEN is_active = 1 THEN 'Active' ELSE 'Inactive' END AS status
FROM student_results
JOIN students ON student_results.student_id = students.id
ORDER BY score DESC;

-- Simple CASE (like switch)
SELECT
  full_name,
```

```
  CASE gender
    WHEN 'Male'   THEN 'Mr.'
    WHEN 'Female' THEN 'Ms.'
    ELSE               'Mx.'
  END AS title
FROM students;
```

Chapter 7 — Aggregate Functions & GROUP BY

Aggregate functions perform calculations across multiple rows and return a single result. They are used with GROUP BY to analyse and summarise data — generating reports, statistics, totals, and averages.

7.1 Core Aggregate Functions

Function	What It Does — Example
COUNT(*)	Count ALL rows, including NULLs. SELECT COUNT(*) FROM students;
COUNT(column)	Count non-NULL values in a column. SELECT COUNT(phone) FROM students;
COUNT(DISTINCT)	Count unique non-NULL values. SELECT COUNT(DISTINCT city) FROM students;
SUM(column)	Sum of all values. SELECT SUM(price) FROM orders;
AVG(column)	Average (mean) of values. SELECT AVG(score) FROM results;
MIN(column)	Smallest value. SELECT MIN(price) FROM products;
MAX(column)	Largest value. SELECT MAX(score) FROM results;
GROUP_CONCAT()	Concatenate values from multiple rows. SELECT GROUP_CONCAT(name) FROM students;

7.2 GROUP BY — Aggregate by Category

```
-- Count students per city
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city
ORDER BY student_count DESC;

-- Average score per course
SELECT
  c.course_name,
  COUNT(r.id)          AS total_students,
  ROUND(AVG(r.score),1) AS avg_score,
  MAX(r.score)         AS highest,
  MIN(r.score)         AS lowest
FROM results r
JOIN courses c ON r.course_id = c.id
GROUP BY c.id, c.course_name
```

```
ORDER BY avg_score DESC;

-- Sales by month
SELECT
  YEAR(order_date)  AS year,
  MONTH(order_date) AS month,
  COUNT(*)          AS total_orders,
  SUM(total_amount) AS revenue
FROM orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY year, month;

-- GROUP_CONCAT — list all courses per student
SELECT
  s.full_name,
  GROUP_CONCAT(c.course_name ORDER BY c.course_name SEPARATOR ', ') AS courses
FROM students s
JOIN enrollments e ON s.id = e.student_id
JOIN courses c ON e.course_id = c.id
GROUP BY s.id, s.full_name;
```

7.3 HAVING — Filter After GROUP BY

```
-- WHERE filters BEFORE grouping; HAVING filters AFTER grouping

-- Cities with more than 5 students
SELECT city, COUNT(*) AS total
FROM students
GROUP BY city
HAVING total > 5
ORDER BY total DESC;

-- Courses where average score is below 60
SELECT course_id, AVG(score) AS avg_score
FROM results
GROUP BY course_id
HAVING avg_score < 60;

-- Combined WHERE and HAVING
SELECT
  c.course_name,
  COUNT(*) AS enrolled
FROM enrollments e
JOIN courses c ON e.course_id = c.id
WHERE e.enrolled >= '2025-01-01'        -- Filter rows BEFORE grouping
GROUP BY c.id, c.course_name
HAVING enrolled >= 10                   -- Filter groups AFTER grouping
ORDER BY enrolled DESC;
```

Chapter 8 — JOINS: INNER, LEFT, RIGHT, SELF & CROSS

Joins combine rows from two or more tables based on a related column between them. They are the most powerful feature of relational databases — allowing you to ask complex questions that span multiple tables.

8.1 INNER JOIN — Only Matching Rows

```
-- Returns only rows where there is a match in BOTH tables
-- Students who have at least one enrollment
SELECT
  s.id,
  s.full_name,
  s.email,
  c.course_name,
  e.grade,
  e.enrolled
FROM students s
INNER JOIN enrollments e ON s.id = e.student_id
INNER JOIN courses c     ON e.course_id = c.id
ORDER BY s.full_name, e.enrolled;

-- INNER JOIN is the default — JOIN and INNER JOIN are identical
FROM students s
JOIN enrollments e ON s.id = e.student_id
```

8.2 LEFT JOIN — All Left Rows + Matches

```
-- Returns ALL rows from the LEFT table, plus matching rows from right.
-- If no match on the right — NULLs are returned for right columns.

-- ALL students, whether they have enrollments or not
SELECT
  s.full_name,
  s.email,
  c.course_name,
  e.grade
FROM students s
LEFT JOIN enrollments e ON s.id = e.student_id
LEFT JOIN courses c     ON e.course_id = c.id
ORDER BY s.full_name;
-- Students with no enrollments show: course_name = NULL, grade = NULL

-- Find students who have NO enrollments at all
```

```
SELECT s.full_name, s.email
FROM students s
LEFT JOIN enrollments e ON s.id = e.student_id
WHERE e.id IS NULL;    -- NULL means no matching row on the right
```

8.3 RIGHT JOIN & FULL JOIN

```
-- RIGHT JOIN — all rows from the RIGHT table + matching from left
-- Less commonly used — you can usually rewrite as a LEFT JOIN
SELECT s.full_name, c.course_name
FROM students s
RIGHT JOIN courses c ON s.id = c.instructor_id;

-- MySQL does NOT support FULL OUTER JOIN directly.
-- Simulate with UNION of LEFT and RIGHT joins:
SELECT s.full_name, c.course_name
FROM students s
LEFT JOIN courses c ON s.id = c.id
UNION
SELECT s.full_name, c.course_name
FROM students s
RIGHT JOIN courses c ON s.id = c.id;
```

8.4 SELF JOIN — A Table Joined to Itself

```
-- Self joins link a table to itself — useful for hierarchical data

-- employees table: id, name, manager_id (references employees.id)
SELECT
  e.full_name   AS employee,
  m.full_name   AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id
ORDER BY m.full_name, e.full_name;

-- Find all students in the same city as 'Lucky Nakola'
SELECT b.full_name, b.city
FROM students a
JOIN students b ON a.city = b.city
WHERE a.full_name = 'Lucky Nakola'
  AND b.full_name != 'Lucky Nakola';
```

8.5 JOIN Summary Table

Join Type	What It Returns
INNER JOIN	Returns rows where there is a match in BOTH tables. Unmatched rows are excluded.
LEFT JOIN	Returns ALL rows from left table. Unmatched right rows return NULL.
RIGHT JOIN	Returns ALL rows from right table. Unmatched left rows return NULL.
CROSS JOIN	Every row in left combined with every row in right (Cartesian product). Rarely used intentionally.
SELF JOIN	A table joined to itself. Used for hierarchical or comparative data.
UNION	Combines results of two SELECT statements vertically. Column count and types must match.
UNION ALL	Same as UNION but keeps duplicate rows (faster than UNION).

Chapter 9 — Subqueries, Views & CTEs

9.1 Subqueries

A subquery is a SELECT statement nested inside another SQL statement. It runs first and its result is used by the outer query.

```
-- Subquery in WHERE — find students who scored above the class average
SELECT full_name, score
FROM results
WHERE score > (SELECT AVG(score) FROM results)  -- scalar subquery
ORDER BY score DESC;

-- Subquery in WHERE with IN
SELECT full_name, email
FROM students
WHERE id IN (
  SELECT student_id FROM enrollments WHERE course_id = 3
);

-- Subquery in FROM (derived table)
SELECT city, avg_score
FROM (
  SELECT s.city, ROUND(AVG(r.score), 1) AS avg_score
  FROM students s
  JOIN results r ON s.id = r.student_id
  GROUP BY s.city
) AS city_averages
WHERE avg_score > 70
ORDER BY avg_score DESC;

-- Correlated subquery — references outer query
SELECT full_name,
  (SELECT COUNT(*) FROM enrollments e WHERE e.student_id = s.id) AS course_count
FROM students s
ORDER BY course_count DESC;

-- EXISTS — check if subquery returns any rows
SELECT full_name FROM students s
WHERE EXISTS (
  SELECT 1 FROM enrollments e WHERE e.student_id = s.id
);
```

9.2 Views

```
-- A VIEW is a saved SELECT query that acts like a virtual table

-- Create a view
CREATE OR REPLACE VIEW v_student_courses AS
SELECT
  s.id          AS student_id,
  s.full_name,
  s.email,
  s.city,
  c.course_name,
  e.grade,
  e.enrolled
FROM students s
JOIN enrollments e ON s.id = e.student_id
JOIN courses c     ON e.course_id = c.id;

-- Use the view exactly like a table
SELECT * FROM v_student_courses WHERE city = 'Nairobi';
SELECT full_name, COUNT(*) FROM v_student_courses GROUP BY full_name;

-- Show all views
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- Drop a view
DROP VIEW IF EXISTS v_student_courses;
```

9.3 Common Table Expressions (CTEs) — WITH Clause

```
-- CTEs make complex queries readable by naming intermediate results

-- Basic CTE
WITH top_scorers AS (
  SELECT student_id, AVG(score) AS avg_score
  FROM results
  GROUP BY student_id
  HAVING avg_score >= 80
)
SELECT s.full_name, s.city, t.avg_score
FROM top_scorers t
JOIN students s ON t.student_id = s.id
ORDER BY t.avg_score DESC;

-- Multiple CTEs
WITH
active_students AS (
  SELECT id, full_name, city FROM students WHERE is_active = 1
),
enrolled_count AS (
```

```
  SELECT student_id, COUNT(*) AS courses FROM enrollments GROUP BY student_id
)
SELECT a.full_name, a.city, COALESCE(e.courses, 0) AS course_count
FROM active_students a
LEFT JOIN enrolled_count e ON a.id = e.student_id
ORDER BY course_count DESC;
```

Chapter 10 — MySQL Data Types: Complete Reference

Choosing the correct data type for each column is one of the most important database design decisions. The right type saves storage space, enforces data validity, and improves query performance.

10.1 Numeric Types

Type	Storage	Range / Notes	Best Used For
TINYINT	1 byte	-128 to 127 / 0 to 255 (UNSIGNED)	Boolean flags, status codes
SMALLINT	2 bytes	-32,768 to 32,767	Age, quantity (small)
MEDIUMINT	3 bytes	-8.3M to 8.3M	Medium-range integers
INT	4 bytes	-2.1B to 2.1B / 0 to 4.2B (UNSIGNED)	Primary keys, counts, IDs
BIGINT	8 bytes	Very large integers (±9.2 quintillion)	Financial transaction IDs
DECIMAL(M,D)	Variable	Exact decimal. M=total digits, D=after point	Prices, money, calculations
FLOAT	4 bytes	Approximate decimal (7 sig digits)	Scientific data, coordinates
DOUBLE	8 bytes	Approximate decimal (15 sig digits)	High-precision measurements
BIT(M)	~M/8 bytes	Bit-field values	Permission bitmasks

10.2 String Types

Type	Storage	Description	Best Used For
CHAR(M)	M bytes (fixed)	Fixed-length string. Padded with spaces. Max 255 chars.	Country codes, status codes
VARCHAR(M)	L+1 or L+2 bytes	Variable-length. Only uses space needed. Max 65,535 chars.	Names, emails, titles
TEXT	L+2 bytes	Up to 65,535 chars. Cannot have DEFAULT value.	Blog posts, descriptions
MEDIUMTEXT	L+3 bytes	Up to 16MB of text.	Large articles, HTML
LONGTEXT	L+4 bytes	Up to 4GB of text.	Massive documents
TINYTEXT	L+1 bytes	Up to 255 chars.	Short descriptions

Type	Storage	Description	Best Used For
ENUM	1-2 bytes	One value from a predefined list.	Status, gender, category
SET	1-8 bytes	Zero or more values from a predefined list.	Multiple-choice flags
BINARY(M)	M bytes	Fixed-length binary string.	Hash values, UUIDs
VARBINARY(M)	L+1 bytes	Variable-length binary string.	Encrypted data
BLOB	L+2 bytes	Binary Large Object — up to 65KB.	Images, files
LONGBLOB	L+4 bytes	Up to 4GB binary data.	Large file storage

10.3 Date & Time Types

Type	Storage	Range / Notes	Best Used For
DATE	3 bytes	YYYY-MM-DD. Range: 1000-01-01 to 9999-12-31.	Birthdays, event dates
TIME	3 bytes	HH:MM:SS. Range: -838:59:59 to 838:59:59.	Duration, time of day
DATETIME	8 bytes	YYYY-MM-DD HH:MM:SS. No timezone. Range: 1000 to 9999.	Event timestamps
TIMESTAMP	4 bytes	YYYY-MM-DD HH:MM:SS. Stored as UTC. Range: 1970–2038.	created_at, updated_at
YEAR	1 byte	YYYY. Range: 1901 to 2155.	Year of graduation

DECIMAL vs FLOAT vs DOUBLE — Critical for Money

FLOAT and DOUBLE store APPROXIMATE values — they have rounding errors.
Example: FLOAT stores 19.99 as something like 19.9899997711181640625
DECIMAL stores EXACT values — always use for money, prices, and financial data.
Rule: Never use FLOAT or DOUBLE for currency. Always use DECIMAL(10,2) or DECIMAL(15,4).
TIMESTAMP vs DATETIME: Use TIMESTAMP for auto-tracking creation/update times.
TIMESTAMP converts to UTC for storage and back to local time on retrieval.
DATETIME stores exactly what you give it — no timezone conversion.

Chapter 11 — Constraints: Rules That Protect Your Data

Constraints are rules applied to columns that MySQL enforces automatically. They prevent bad, inconsistent, or duplicate data from being inserted or updated — protecting your database integrity at the engine level.

11.1 All Constraints Explained

```
CREATE TABLE employees (
  -- PRIMARY KEY: unique identifier, not null, auto-increment
  id           INT           NOT NULL AUTO_INCREMENT,

  -- NOT NULL: column must always have a value
  full_name    VARCHAR(100)  NOT NULL,

  -- UNIQUE: no two rows can have the same value
  email        VARCHAR(150)  NOT NULL UNIQUE,

  -- DEFAULT: value used when none is provided
  department   VARCHAR(80)   NOT NULL DEFAULT 'General',
  status       TINYINT(1)    NOT NULL DEFAULT 1,
  joined_date  DATE          NOT NULL DEFAULT (CURRENT_DATE),

  -- CHECK: value must satisfy a condition (MySQL 8.0.16+)
  salary       DECIMAL(12,2) NOT NULL CHECK (salary > 0),
  age          TINYINT       NULL     CHECK (age >= 18 AND age <= 70),

  -- FOREIGN KEY: references another table's primary key
  department_id INT          NULL,

  PRIMARY KEY (id),
  CONSTRAINT fk_dept FOREIGN KEY (department_id)
    REFERENCES departments(id)
    ON DELETE SET NULL
    ON UPDATE CASCADE
) ENGINE=InnoDB;
```

11.2 Foreign Key Actions

Action	What Happens to Child Rows When Parent Changes
RESTRICT	Default. Prevents deleting/updating a parent row if child rows reference it. Most protective.
CASCADE	Automatically deletes/updates child rows when parent row is deleted/updated. Use with care.
SET NULL	Sets the foreign key column to NULL when parent is deleted/updated. Column must be nullable.
SET DEFAULT	Sets FK to its default value when parent changes. Rarely used in MySQL/InnoDB.
NO ACTION	Same as RESTRICT in MySQL. Check is deferred (if supported).

11.3 Adding & Removing Constraints

```
-- Add NOT NULL constraint (by modifying the column)
ALTER TABLE students MODIFY COLUMN phone VARCHAR(20) NOT NULL;

-- Add UNIQUE constraint
ALTER TABLE students ADD CONSTRAINT uq_student_no UNIQUE (student_no);

-- Add CHECK constraint
ALTER TABLE products ADD CONSTRAINT chk_price CHECK (price >= 0);

-- Add DEFAULT value
ALTER TABLE students ALTER COLUMN city SET DEFAULT 'Nairobi';

-- Remove DEFAULT
ALTER TABLE students ALTER COLUMN city DROP DEFAULT;

-- Drop UNIQUE constraint (treated as an index)
ALTER TABLE students DROP INDEX uq_student_no;

-- Drop FOREIGN KEY
ALTER TABLE enrollments DROP FOREIGN KEY fk_enrollment_student;

-- Drop CHECK constraint
ALTER TABLE products DROP CHECK chk_price;

-- View all constraints on a table
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = 'school_db' AND TABLE_NAME = 'students';
```

Chapter 12 — Indexes, Performance & Query Optimisation

An index is a separate data structure (usually a B-tree) that MySQL maintains alongside a table to speed up data retrieval. Without an index, MySQL must scan every row in a table (full table scan) to find matching rows. With an index, it can jump directly to the right location — like using a book's index instead of reading every page.

12.1 Types of Indexes

Index Type	Purpose & Notes
PRIMARY KEY	Unique, not null. Every InnoDB table is physically ordered by the PRIMARY KEY (clustered index). Auto-created.
UNIQUE	Ensures no duplicate values. Can contain one NULL. Auto-created for UNIQUE constraints.
INDEX (KEY)	General-purpose index. Speeds up WHERE, JOIN, and ORDER BY on the indexed column.
FULLTEXT	Used for full-text searches with MATCH...AGAINST. Only on TEXT, CHAR, VARCHAR columns.
SPATIAL	For geospatial data (GIS). Used with geometry data types.
Composite	Index on multiple columns together. The order of columns matters significantly.
Covering	Index that includes all columns needed by a query — eliminates need to read the table itself.

12.2 Creating & Managing Indexes

```
-- Create index during table creation
CREATE TABLE orders (
  id          INT NOT NULL AUTO_INCREMENT,
  customer_id INT NOT NULL,
  status      VARCHAR(20),
  order_date  DATE NOT NULL,
  total       DECIMAL(10,2),
  PRIMARY KEY (id),
  INDEX idx_customer  (customer_id),
  INDEX idx_date      (order_date),
  INDEX idx_status_date (status, order_date)    -- Composite index
);

-- Add index to existing table
```

```
CREATE INDEX idx_city         ON students (city);
CREATE INDEX idx_name_city    ON students (full_name, city);  -- Composite
CREATE UNIQUE INDEX uq_email  ON students (email);
CREATE FULLTEXT INDEX ft_name ON students (full_name);

-- Drop an index
DROP INDEX idx_city ON students;
ALTER TABLE students DROP INDEX idx_city;

-- View all indexes on a table
SHOW INDEXES FROM students;
```

12.3 EXPLAIN — Analysing Query Performance

```
-- EXPLAIN shows how MySQL executes a query
EXPLAIN SELECT * FROM students WHERE city = 'Nairobi';

-- What to look for in EXPLAIN output:
--  type:  'ALL' = full table scan (BAD on large tables)
--         'ref' or 'eq_ref' = index lookup (GOOD)
--         'const' = single row by primary key (BEST)
--  key:   which index MySQL used (NULL = no index used)
--  rows:  estimated rows MySQL must examine
--  Extra: 'Using index' = covering index (very fast)

-- EXPLAIN ANALYZE (MySQL 8.0.18+) — actually runs the query and shows real stats
EXPLAIN ANALYZE
SELECT s.full_name, COUNT(e.id) AS enrollments
FROM students s
LEFT JOIN enrollments e ON s.id = e.student_id
GROUP BY s.id
ORDER BY enrollments DESC;
```

12.4 Query Optimisation Best Practices

- Always index columns used in WHERE, JOIN ON, and ORDER BY clauses.
- Use SELECT col1, col2 instead of SELECT * — return only the columns you need.
- Avoid functions on indexed columns in WHERE: WHERE YEAR(created_at)=2025 prevents index use. Use WHERE created_at BETWEEN instead.
- Use LIMIT to restrict result sets — especially for large tables.
- For composite indexes, the leftmost column must be in the WHERE clause.
- Avoid SELECT DISTINCT unnecessarily — it forces a sort operation.
- Use EXISTS instead of COUNT() when checking whether rows exist.
- Normalise your schema to reduce data redundancy.
- Run ANALYZE TABLE tablename; periodically to update index statistics.

Chapter 13 — Stored Procedures, Functions & Triggers

13.1 Stored Procedures

A stored procedure is a named block of SQL code stored in the database and executed on demand. It can accept parameters, contain logic (loops, conditionals), and execute multiple SQL statements.

```
DELIMITER $$

CREATE PROCEDURE sp_enroll_student(
  IN  p_student_id INT,
  IN  p_course_id  INT,
  OUT p_result     VARCHAR(100)
)
BEGIN
  -- Check if student exists
  IF NOT EXISTS (SELECT 1 FROM students WHERE id = p_student_id) THEN
    SET p_result = 'ERROR: Student not found.';
  -- Check if already enrolled
  ELSEIF EXISTS (
    SELECT 1 FROM enrollments WHERE student_id=p_student_id AND
course_id=p_course_id
  ) THEN
    SET p_result = 'ERROR: Already enrolled in this course.';
  ELSE
    -- Enroll the student
    INSERT INTO enrollments (student_id, course_id, enrolled)
    VALUES (p_student_id, p_course_id, CURRENT_DATE);

    SET p_result = CONCAT('SUCCESS: Enrolled in course ', p_course_id);
  END IF;
END$$

DELIMITER ;

-- Call the procedure
CALL sp_enroll_student(1, 3, @result);
SELECT @result;

-- List all stored procedures
SHOW PROCEDURE STATUS WHERE Db = 'school_db';

-- Drop a procedure
DROP PROCEDURE IF EXISTS sp_enroll_student;
```

13.2 Stored Functions

```
DELIMITER $$

CREATE FUNCTION fn_get_grade(p_score DECIMAL(5,2))
RETURNS CHAR(2)
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE grade CHAR(2);
  IF    p_score >= 90 THEN SET grade = 'A';
  ELSEIF p_score >= 75 THEN SET grade = 'B';
  ELSEIF p_score >= 60 THEN SET grade = 'C';
  ELSEIF p_score >= 50 THEN SET grade = 'D';
  ELSE                      SET grade = 'F';
  END IF;
  RETURN grade;
END$$

DELIMITER ;

-- Use the function in a SELECT
SELECT full_name, score, fn_get_grade(score) AS grade
FROM student_results
ORDER BY score DESC;
```

13.3 Triggers

```
-- A trigger fires automatically BEFORE or AFTER an INSERT, UPDATE, or DELETE

DELIMITER $$

-- Log every student update to an audit table
CREATE TRIGGER trg_students_after_update
AFTER UPDATE ON students
FOR EACH ROW
BEGIN
  INSERT INTO audit_log (table_name, action, record_id, old_value, new_value,
changed_at)
  VALUES (
    'students',
    'UPDATE',
    OLD.id,
    CONCAT('name:', OLD.full_name, ', city:', OLD.city),
    CONCAT('name:', NEW.full_name, ', city:', NEW.city),
    NOW()
  );
END$$

-- Set a default student number before insert
```

```
CREATE TRIGGER trg_students_before_insert
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
  IF NEW.student_no IS NULL OR NEW.student_no = '' THEN
    SET NEW.student_no = CONCAT('STU', LPAD((SELECT COUNT(*)+1 FROM students), 5,
'0'));
  END IF;
END$$

DELIMITER ;

-- View all triggers
SHOW TRIGGERS FROM school_db;

-- Drop a trigger
DROP TRIGGER IF EXISTS trg_students_after_update;
```

Chapter 14 — Transactions, ACID Properties & Locking

A transaction is a unit of work that must succeed completely or fail completely. It is the mechanism that keeps a database consistent when multiple operations must happen together — like transferring money from one account to another.

14.1 ACID Properties

Property	What It Guarantees
Atomicity	All operations in a transaction succeed OR all are rolled back. No partial updates. If deducting from one account fails, the credit to the other is also reversed.
Consistency	A transaction brings the database from one valid state to another. Constraints and rules are always satisfied after a transaction.
Isolation	Concurrent transactions do not interfere with each other. Each transaction sees a consistent view of the data — as if it were the only transaction running.
Durability	Once a transaction is committed, it is permanently saved — even if the server crashes immediately after. Changes are written to disk.

14.2 Transaction Control Statements

```
-- Transactions only work with InnoDB engine (default in MySQL 5.5+)

-- Start a transaction
START TRANSACTION;

-- Example: Transfer KES 5,000 from account 101 to account 202
START TRANSACTION;

UPDATE accounts SET balance = balance - 5000 WHERE id = 101;
UPDATE accounts SET balance = balance + 5000 WHERE id = 202;

-- Check no one went below zero
SELECT balance FROM accounts WHERE id = 101;

-- If everything looks good — permanently save both changes
COMMIT;

-- If something went wrong — undo EVERYTHING back to before START TRANSACTION
ROLLBACK;
```

```
-- SAVEPOINT — create a named checkpoint within a transaction
START TRANSACTION;
INSERT INTO orders (customer_id, total) VALUES (5, 1500.00);
SAVEPOINT order_created;

INSERT INTO order_items (order_id, product_id, qty) VALUES (LAST_INSERT_ID(), 12,
2);
-- Something went wrong with items...
ROLLBACK TO SAVEPOINT order_created;  -- Only undo back to the savepoint
COMMIT;                               -- The order is still created

-- AutoCommit — by default MySQL auto-commits every statement
-- Turn off to manage transactions manually:
SET autocommit = 0;
-- ... run statements ...
COMMIT;
SET autocommit = 1;
```

Chapter 15 — Database Design: Normalization & ER Diagrams

Good database design is the foundation of every fast, reliable, and maintainable application. Normalisation is the process of organising a database to reduce redundancy, eliminate anomalies, and ensure data integrity.

15.1 Why Normalization Matters

Benefit	Explanation
Reduces redundancy	The same data is stored in only one place. Changing a value updates it everywhere automatically.
Prevents update anomalies	If a student's city is stored in 50 rows, changing it requires 50 updates — one miss = inconsistency.
Prevents insertion anomalies	In a bad design, you might be unable to add a course without assigning a student to it.
Prevents deletion anomalies	Deleting a student might accidentally delete course data if it was stored together.
Improves integrity	Rules and constraints are easier to enforce when data is properly structured.

15.2 Normal Forms

First Normal Form (1NF)

A table is in 1NF when: every column contains atomic (single, indivisible) values, every row is unique (has a primary key), and there are no repeating groups of columns.

```
-- ❌ NOT in 1NF — courses are a comma list in one column
id | student_name | courses
1  | Lucky        | PHP, MySQL, JavaScript

-- ☑ 1NF — each course in its own row
id | student_name | course
1  | Lucky        | PHP
1  | Lucky        | MySQL
1  | Lucky        | JavaScript

-- Better: separate students and courses into different tables
```

Second Normal Form (2NF)

A table is in 2NF when it is in 1NF AND every non-key column is fully dependent on the ENTIRE primary key (not just part of it). This only matters for composite primary keys.

```
-- ❌ NOT in 2NF — course_name depends only on course_id, not the full PK
PK: (student_id, course_id)
student_id | course_id | course_name | grade
1          | 3         | PHP Dev     | A

-- ☑ 2NF — split course_name into a separate courses table
-- enrollments: student_id, course_id, grade
-- courses:     id, course_name
```

Third Normal Form (3NF)

A table is in 3NF when it is in 2NF AND no non-key column depends on another non-key column (no transitive dependencies).

```
-- ❌ NOT in 3NF — city depends on zip_code, which is not the primary key
id | student_name | zip_code | city
1  | Lucky        | 10100    | Nairobi

-- ☑ 3NF — separate zip_code/city into a locations table
-- students:  id, student_name, zip_code
-- locations: zip_code, city, county
```

15.3 Entity-Relationship (ER) Diagram Description

An ER Diagram visually shows tables, their columns, and the relationships between them. For the school database:

```
SCHOOL DATABASE ER STRUCTURE
================================

[ USERS ]                    [ STUDENTS ]
  id (PK)                      id (PK)
  name                         student_no (UNIQUE)
  email (UNIQUE)               full_name
  password_hash                email (UNIQUE)
  role                         city
  created_at                   user_id (FK → users.id)
                               created_at
          |
          | 1:1
          ↓

[ COURSES ]                  [ INSTRUCTORS ]
  id (PK)                      id (PK)
  course_name                  full_name
  description                  email
```

```
  duration_weeks              specialisation
  fee                         user_id (FK → users.id)
  instructor_id (FK → instructors.id)

  ← 1 instructor teaches many courses (1:N)

[ ENROLLMENTS ]  ← junction table for Students ↔ Courses (M:N)
  id (PK)
  student_id  (FK → students.id)
  course_id   (FK → courses.id)
  enrolled    (DATE)
  grade       (CHAR)
  UNIQUE (student_id, course_id)  ← no duplicate enrollments

[ RESULTS ]
  id (PK)
  student_id  (FK → students.id)
  course_id   (FK → courses.id)
  score       (DECIMAL)
  exam_date   (DATE)
  created_at  (TIMESTAMP)
```

Chapter 16 — Security, Backup & Database Administration

16.1 User Management & Privileges

```
-- Create a new MySQL user
CREATE USER 'school_app'@'localhost'  IDENTIFIED BY 'StrongPassword123!';
CREATE USER 'school_app'@'%'          IDENTIFIED BY 'StrongPassword123!';
-- @'localhost' = only local connections; @'%' = any host

-- Grant privileges
GRANT SELECT, INSERT, UPDATE, DELETE ON school_db.* TO 'school_app'@'localhost';
GRANT ALL PRIVILEGES ON school_db.*  TO 'school_admin'@'localhost';
GRANT SELECT ON school_db.students  TO 'readonly_user'@'localhost';

-- Show a user's privileges
SHOW GRANTS FOR 'school_app'@'localhost';

-- Revoke privileges
REVOKE DELETE ON school_db.* FROM 'school_app'@'localhost';

-- Apply privilege changes immediately
FLUSH PRIVILEGES;

-- Change a user's password
ALTER USER 'school_app'@'localhost' IDENTIFIED BY 'NewPassword456!';

-- Drop a user
DROP USER IF EXISTS 'school_app'@'localhost';

-- List all users
SELECT user, host FROM mysql.user;
```

16.2 Backup & Restore

```
-- === BACKUP with mysqldump ===

-- Backup a single database
mysqldump -u root -p school_db > school_db_backup_2025-06-01.sql

-- Backup with timestamps (Windows)
mysqldump -u root -p school_db > school_db_%date:~-4,4%%date:~-10,2%%date:~-
7,2%.sql

-- Backup specific tables only
mysqldump -u root -p school_db students courses enrollments > tables_backup.sql
```

```
-- Backup all databases
mysqldump -u root -p --all-databases > all_databases.sql

-- Backup with structure only (no data) — good for dev environment setup
mysqldump -u root -p --no-data school_db > schema_only.sql

-- Backup data only (no CREATE TABLE statements)
mysqldump -u root -p --no-create-info school_db > data_only.sql

-- Compressed backup (much smaller file)
mysqldump -u root -p school_db | gzip > school_db_backup.sql.gz

-- === RESTORE ===

-- Restore a database (database must exist first)
mysql -u root -p school_db < school_db_backup_2025-06-01.sql

-- Create database and restore
mysql -u root -p -e 'CREATE DATABASE school_db'
mysql -u root -p school_db < school_db_backup.sql

-- Restore compressed backup
gunzip < school_db_backup.sql.gz | mysql -u root -p school_db
```

16.3 Security Best Practices

Practice	Why It Matters & How to Apply
Use prepared statements	NEVER build SQL with string concatenation from user input. SQL injection is the #1 database attack.
Principle of least privilege	Give each app user only the permissions it needs. Web app needs SELECT/INSERT/UPDATE/DELETE — NOT DROP or CREATE.
Never use root in apps	Create a dedicated database user for each application. Never connect as root from application code.
Strong passwords	Use long, complex passwords for all MySQL users. Store in environment variables, not in code.
Limit host access	Use @'localhost' instead of @'%' whenever possible to restrict which hosts can connect.
Encrypt connections	Enable SSL/TLS for database connections, especially over networks.
Regular backups	Daily automated backups. Test restores regularly. Off-site or cloud backup copies.
Audit logging	Enable MySQL general log or binary log for auditing. Know who changed what and when.

Practice	Why It Matters & How to Apply
Update MySQL regularly	Security patches are released regularly. Keep MySQL version current.
Remove test databases	Run 'mysql_secure_installation' after install to remove the anonymous test database.

Chapter 17 — MySQL with PHP: PDO & MySQLi Integration

PHP and MySQL are the most powerful combination in web development. This chapter shows how to connect, query, and manage MySQL data from PHP applications — using both PDO and MySQLi with prepared statements.

17.1 Complete PDO Database Class

```php
<?php
// Database.php — reusable PDO wrapper class
class Database {
    private static ?PDO $instance = null;

    public static function connect(): PDO {
        if (self::$instance === null) {
            $dsn = sprintf('mysql:host=%s;dbname=%s;charset=utf8mb4',
                getenv('DB_HOST') ?: 'localhost',
                getenv('DB_NAME') ?: 'school_db'
            );
            self::$instance = new PDO(
                $dsn,
                getenv('DB_USER') ?: 'root',
                getenv('DB_PASS') ?: '',
                [
                    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                    PDO::ATTR_EMULATE_PREPARES   => false,
                    PDO::MYSQL_ATTR_FOUND_ROWS   => true,
                ]
            );
        }
        return self::$instance;
    }
}

// Usage
$pdo = Database::connect();
?>
```

17.2 Full CRUD with PDO

```
<?php
class StudentRepository {
    public function __construct(private PDO $pdo) {}

    // CREATE
    public function create(array $data): int {
        $sql  = 'INSERT INTO students (student_no, full_name, email, city) VALUES
(:no, :name, :email, :city)';
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute([':no'=>$data['student_no'], ':name'=>$data['full_name'],
                        ':email'=>$data['email'], ':city'=>$data['city']]);
        return (int)$this->pdo->lastInsertId();
    }

    // READ ALL
    public function getAll(int $limit=50, int $offset=0): array {
        $stmt = $this->pdo->prepare('SELECT * FROM students ORDER BY full_name
LIMIT :limit OFFSET :offset');
        $stmt->bindValue(':limit',  $limit,  PDO::PARAM_INT);
        $stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
        $stmt->execute();
        return $stmt->fetchAll();
    }

    // READ ONE
    public function getById(int $id): ?array {
        $stmt = $this->pdo->prepare('SELECT * FROM students WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->fetch() ?: null;
    }

    // UPDATE
    public function update(int $id, array $data): int {
        $sql  = 'UPDATE students SET full_name=:name, email=:email, city=:city
WHERE id=:id';
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute([':name'=>$data['full_name'], ':email'=>$data['email'],
                        ':city'=>$data['city'], ':id'=>$id]);
        return $stmt->rowCount();
    }

    // DELETE
    public function delete(int $id): int {
        $stmt = $this->pdo->prepare('DELETE FROM students WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->rowCount();
    }

    // TRANSACTION EXAMPLE
```

```
    public function transferEnrollment(int $studentId, int $fromCourse, int
$toCourse): void {
        $this->pdo->beginTransaction();
        try {
            $del = $this->pdo->prepare('DELETE FROM enrollments WHERE
student_id=:s AND course_id=:c');
            $del->execute([':s'=>$studentId, ':c'=>$fromCourse]);
            $ins = $this->pdo->prepare('INSERT INTO enrollments (student_id,
course_id) VALUES (:s, :c)');
            $ins->execute([':s'=>$studentId, ':c'=>$toCourse]);
            $this->pdo->commit();
        } catch (Exception $e) {
            $this->pdo->rollBack();
            throw $e;
        }
    }
}

// Usage
$repo = new StudentRepository(Database::connect());
$id   = $repo->create(['student_no'=>'STU010','full_name'=>'Test
User','email'=>'t@example.com','city'=>'Nyeri']);
$all  = $repo->getAll(10, 0);
?>
```

Chapter 18 — Comprehensive Quiz & Practical Challenges

Test your complete understanding of MySQL and database management. Attempt every section independently before reviewing answers.

Section A — Multiple Choice (25 questions — 1 mark each)

1. What does SQL stand for?

   a) Structured Query Language b) Simple Query Language c) Sequential Query Logic d) Structured Queue Language

2. Which SQL command is used to retrieve data from a table?

   a) GET b) FETCH c) SELECT d) RETRIEVE

3. Which category of SQL includes CREATE, ALTER, and DROP?

   a) DML — Data Manipulation Language b) DCL — Data Control Language
   c) DDL — Data Definition Language d) TCL — Transaction Control Language

4. What does a PRIMARY KEY guarantee?

   a) The column can contain NULL values
   b) Every row has a unique, non-null identifier
   c) The column is automatically indexed but can contain duplicates
   d) The column references a key in another table

5. What is the difference between DELETE and TRUNCATE?

   a) DELETE removes the table; TRUNCATE removes only the data
   b) TRUNCATE removes rows instantly and resets AUTO_INCREMENT; DELETE is row-by-row and can be rolled back
   c) They are identical — just different syntax
   d) TRUNCATE can use a WHERE clause; DELETE cannot

6. Which JOIN returns ALL rows from the left table even when there is no match on the right?

   a) INNER JOIN b) CROSS JOIN c) RIGHT JOIN d) LEFT JOIN

7. What is the purpose of a FOREIGN KEY constraint?

   a) To ensure a column has no NULL values
   b) To prevent duplicate values in a column
   c) To enforce a relationship between two tables and maintain referential integrity
   d) To speed up queries on the referenced column

8. What does GROUP BY do in a SQL query?

   a) Sorts the result set in ascending order
   b) Groups rows that have the same values in specified columns for use with aggregate functions
   c) Filters rows before they are processed by aggregate functions
   d) Joins two tables on a common column

9. What is the difference between WHERE and HAVING?

   a) WHERE filters individual rows before grouping; HAVING filters groups after GROUP BY
   b) HAVING filters rows before grouping; WHERE filters after GROUP BY
   c) They are interchangeable — both filter rows
   d) WHERE works only with numbers; HAVING works with strings

10. Which data type should ALWAYS be used to store monetary values like prices?

    a) FLOAT b) DOUBLE c) INT d) DECIMAL

11. What is a database index?

    a) A backup copy of a table stored on a separate server
    b) A data structure that speeds up data retrieval at the cost of additional storage
    c) A list of all users who have access to the database
    d) A constraint that prevents duplicate values in a column

12. What does EXPLAIN do in MySQL?

    a) Shows the SQL syntax definition of a table
    b) Shows how MySQL plans to execute a query — which indexes it will use
    c) Explains the meaning of a MySQL error code
    d) Displays all stored procedures in a database

13. What does ON DELETE CASCADE mean on a FOREIGN KEY?

    a) Prevent deleting a parent row if child rows exist
    b) Set child foreign key to NULL when parent is deleted
    c) Automatically delete all child rows when the parent row is deleted
    d) Rename the child rows when the parent row is deleted

14. What is the correct MySQL function to get the current date and time?

    a) DATE() b) GETDATE() c) SYSDATE() d) NOW()

15. What is a database VIEW?

    a) A permanent copy of a table's data stored in memory
    b) A saved SELECT query that acts as a virtual table
    c) A visualisation tool built into phpMyAdmin
    d) A type of index that improves GROUP BY performance

16. What do the ACID properties stand for?

    a) Atomicity, Consistency, Isolation, Durability
    b) Authenticity, Concurrency, Integrity, Dependency
    c) Atomicity, Correctness, Indexing, Distribution
    d) Availability, Concurrency, Isolation, Definition

17. What happens when you run ROLLBACK in a transaction?

    a) All changes since the last COMMIT are permanently saved
    b) The database connection is closed
    c) All changes made since START TRANSACTION are undone
    d) The transaction is paused until a COMMIT is issued

18. Which command shows the structure (columns, types, constraints) of a table?

    a) SHOW TABLE students; b) DESCRIBE students; c) STRUCTURE students; d) COLUMNS students;

19. What is Third Normal Form (3NF)?

    a) Every column contains atomic values and there are no repeating groups
    b) All non-key columns are fully dependent on the primary key (no partial dependencies)
    c) No non-key column depends on another non-key column (no transitive dependencies)
    d) Every table must have exactly three columns as its composite primary key

20. Which MySQL tool provides a web-based graphical interface for managing databases?

    a) MySQL Workbench b) DBeaver c) phpMyAdmin d) HeidiSQL

21. What is the purpose of a junction (pivot) table in a Many-to-Many relationship?

    a) To store large binary files across multiple tables
    b) To resolve a M:N relationship by creating two 1:N relationships with a linking table
    c) To speed up queries by combining frequently joined tables
    d) To create a backup of both related tables

22. What does the CONCAT() function do in MySQL?

    a) Counts the number of rows in a result set
    b) Joins two or more strings together into one string
    c) Converts a string to uppercase
    d) Calculates the total of a numeric column

23. Which SQL clause is used to remove duplicate rows from a SELECT result?

    a) UNIQUE b) NO REPEAT c) DISTINCT d) DIFFERENT

24. What privilege level is recommended for a PHP web application's database user?

    a) ALL PRIVILEGES on all databases — gives maximum flexibility
    b) SUPER privilege — required for stored procedures

c) SELECT, INSERT, UPDATE, DELETE on the application's database only
d) No privileges — the app should connect as root for reliability

25. Which mysqldump flag creates a backup that includes only the table structure, NOT the data?

a) --no-data b) --structure-only c) --schema d) --empty

Section B — Short Answer (10 questions — 3 marks each)

1. Explain the difference between a PRIMARY KEY and a FOREIGN KEY. How do they work together to create a relationship between two tables? Give a concrete example using students and enrollments.

2. What is SQL injection? Give an example of a vulnerable PHP+MySQL code snippet and then show the safe version using a prepared statement. Explain why the safe version prevents the attack.

3. Explain the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN. For each one, describe what rows are returned when there is NO matching row on the other table. Give a practical use case for each.

4. What is database normalisation and why is it important? Briefly explain First, Second, and Third Normal Form (1NF, 2NF, 3NF) with a simple example showing the progression from an unnormalised table to 3NF.

5. What are the ACID properties of a database transaction? Explain each property with a real-world banking example involving a money transfer between two accounts.

6. What is the purpose of a database index? What are the benefits and drawbacks of adding indexes? When should you add an index and when should you avoid it?

7. Explain the difference between a Stored Procedure and a Stored Function in MySQL. What can each do? Write a simple stored function that takes a student's score and returns their letter grade.

8. What is the difference between DATETIME and TIMESTAMP data types in MySQL? In what situation would you use each one? What happens to TIMESTAMP values when the server timezone changes?

9. What is a CTE (Common Table Expression) and how does it differ from a subquery in the FROM clause? Write a CTE query that finds all students whose average score is above the overall class average.

10. Describe five MySQL database security best practices. For each one, explain what vulnerability it protects against and how to implement it properly.

Section C — Practical Coding Challenges

Challenge 1 — School Database Design & SQL:

Design and implement a complete school database called 'academy_db' with these requirements: (a) Create a 'users' table for authentication: id, name, email (unique), password_hash, role (ENUM: admin/instructor/student), is_active, created_at. (b) Create a 'courses' table: id, course_code (unique), course_name, description, duration_weeks, fee (DECIMAL), instructor_id (FK to users), max_students, is_published, created_at. (c) Create an 'enrollments' junction table (M:N between students and courses): id, user_id (FK), course_id (FK), enrolled_date, status (ENUM: active/completed/dropped), prevent duplicate enrollment with a UNIQUE constraint. (d) Create a 'results' table: id, user_id (FK), course_id (FK), score (DECIMAL 5,2), grade (CHAR 2), exam_date, created_at. (e) Write a VIEW called v_course_stats that shows: course name, instructor name, total enrolled, average score, highest score, lowest score. (f) Write a stored procedure sp_get_student_report that accepts a student user_id and returns all their courses with scores and grades. (g) Write 5 analytical queries: top 3 courses by enrollment count; all students who passed (score >= 50) with their grade; monthly enrollment count for 2025; instructors whose students average above 75; and students enrolled in more than 2 courses.

Challenge 2 — Query Writing Practice:

Using the school database above, write SQL for each of the following: (a) Find all students (name, email) who have NOT enrolled in any course. (b) Find the 5 most recent enrollments showing student name, course name, and enrolled date. (c) Calculate each student's total fees paid based on courses they are enrolled in. (d) Find pairs of students who are enrolled in the same course (SELF JOIN on enrollments). (e) List all courses along with their enrollment count, even courses with zero enrollments. (f) Find the student with the highest average score across all their courses. (g) For each city, show the count of students and their average score, but only include cities with 3 or more students. (h) Update all grades in the results table based on score using a CASE expression.

Challenge 3 — PHP + MySQL Complete Application:

Build a PHP + MySQL mini Course Management System using PDO: (a) config/database.php — PDO connection singleton class. (b) CourseRepository.php — class with methods: getAll(), getById(), create(), update(), delete(), getWithStats() (joins to get enrollment count and average score). (c) StudentRepository.php — getAll(), getById(), create(), enroll(studentId, courseId),

getEnrollments(studentId). (d) index.php — display all courses in a table with Name, Instructor, Enrolled Count, Average Score, Fee. Link each to a details page. (e) course_detail.php — show full course details and a list of all enrolled students with their scores. (f) enroll.php — form to enroll a student in a course. Handle duplicate enrollment gracefully. (g) Use a transaction when creating an enrollment and recording an initial results entry. (h) All queries must use prepared statements. All output must use htmlspecialchars(). Display friendly error messages — no raw SQL errors to the user.

Section A — Answer Key

Multiple Choice Answers

1. a) Structured Query Language 2. c) SELECT
3. c) DDL — Data Definition Language 4. b) Unique, non-null identifier per row
5. b) TRUNCATE resets AUTO_INCREMENT; DELETE can be rolled back
6. d) LEFT JOIN 7. c) Enforces referential integrity
8. b) Groups rows for aggregate functions 9. a) WHERE before grouping; HAVING after
10. d) DECIMAL 11. b) Speeds up retrieval — extra storage cost
12. b) Shows query execution plan 13. c) Auto-delete child rows
14. d) NOW() 15. b) Saved SELECT — virtual table
16. a) Atomicity, Consistency, Isolation, Durability
17. c) All changes since START TRANSACTION are undone
18. b) DESCRIBE students; 19. c) No transitive dependencies
20. c) phpMyAdmin 21. b) Two 1:N relationships via a linking table
22. b) Joins two or more strings together 23. c) DISTINCT
24. c) SELECT, INSERT, UPDATE, DELETE only 25. a) --no-data

Data is the new oil. SQL is how you refine it.

Get Techy With Lucky | Tech Pulse Insider | Instructor: Lucky Nakola